

The ZC-DMC Hash Function

Siwei Sun, Shun Li, Zhiyu Zhang, Fangyu Zheng, Guozhen Liu,
Yong Liu, Jianhua Wang, Zhen Qin, Ruixi Lv author@mail.com

01/05/2026

Contents

Chapter 1 Specification	5
1.1 The ZC-m Permutations	5
1.1.1 Basic Components of ZC-1280	5
1.1.2 Basic Components of ZC-1536	8
1.1.3 Round Function	9
1.2 The Construction SPONGE-DM ^c	10
1.3 The Hash Functions	13
Chapter 2 Design Rationale	17
2.1 Design of the Construction	17
2.2 Design of the Permutation	18
2.2.1 Choice of the Round Constants	18
2.2.2 Choice of the Substitution Layer	18
2.2.3 Choice of the Linear Diffusion Layer	19
Chapter 3 Security Analysis	21
3.1 Security Analysis of the Construction SPONGE-DM ^c	21
3.1.1 Dedicated attack on reduced feed-forward SPONGE-DM ^c	21
3.2 Security Evaluation of the Permutation	21
3.3 Security Evaluation of the Hash Functions	22
3.3.1 Collision Resistance	22
3.3.2 Preimage Resistance	23
3.3.3 Second-Preimage Resistance	23
3.4 Quantum Security	24
3.4.1 Collision Resistance	24
3.4.2 Preimage Resistance	24
3.4.3 Second-Preimage Resistance	24
3.5 Other Security Properties	25
3.5.1 Length Extension Attack Resistance	25
3.5.2 Zero-sum Distinguisher	25
3.5.3 Statistical Randomness	25
Chapter 4 Performance Evaluation	29
4.1 Performance analysis (design principles)	29
4.1.1 Elementary operations and round-function cost	29
4.1.2 Mode-level overhead	30
4.2 Performance testing (software implementations)	30

4.2.1	Implementation variants and platform configuration	30
4.2.2	Benchmark methodology	31
4.2.3	Benchmark results	31
Chapter 5	Feature Overview	33
5.1	Novelty and Comparison to Keccak-p and SHA-3	33
5.1.1	Specification and Design Choices	33
5.1.2	Security Highlights	33
5.1.3	Efficiency–Security Trade-off via DM^c	34
Bibliography		34

Chapter 1

Specification

The hash construction specified in this document is based on the **SPONGE-DM^c** mode introduced in [1], to appear in IACR CRYPTO 2026.

This chapter specifies the permutations **ZC-1280**, **ZC-1536**, the construction **SPONGE-DM^c**, and the hash function **ZC-DMC-m-n** ($m \in \{1280, 1536\}$, $n \in \{512, 768, 1024\}$).

1.1 The ZC-m Permutations

The **ZC-m** is a family of permutations parameterized by the number of rounds n_r , denoted by **ZC-m** $[n_r]$. The family includes two variants: the 1280-bit permutation **ZC-1280** and the 1536-bit permutation **ZC-1536**, defined in Algorithm 1 and Algorithm 2. It is a generalization of the “XOODOO” that allows for 3D structure. The **ZC-m** permutations consist of 12 rounds, each round transformation p consists of five steps ι , χ , ρ , θ , and ρ , applied in the following order:

$$p = \rho \circ \theta \circ \rho \circ \chi \circ \iota.$$

The χ functions are based on the 3/5-bit χ mapping applied to each column, which provides good resistance against differential and linear cryptanalysis. The linear layer consists of the column parity mixer θ and the plane shift ρ , designed to provide good diffusion properties while being efficient to implement. Overall, the **ZC-m** permutations are designed to be secure against known cryptanalytic attacks while being efficient to implement in both software and hardware.

1.1.1 Basic Components of ZC-1280

State The state size of **ZC-1280** is 1280 bits, which can be organized into 5 equally sized horizontal *planes*, each plane consists of 4 parallel 64-bit *lanes*, as shown in Figure 1.1. Equivalently, the **ZC-1280** state S may be viewed as a collection of 256 *columns* of 5 bits, arranged in a 4×64 array $S[x][y][z]$. Planes are indexed by the coordinate y , with plane $y = 0$ at the bottom and plane $y = 4$ at the top. Within each lane, bits are indexed by z . Lanes within a plane are indexed by x , so the position of a lane in the state is uniquely identified by the coordinates (x, y) . Consequently, each bit of the state is indexed by the triplet (x, y, z) , while columns are indexed by (x, z) . We further define *Sheets* as vertical stacks of 5 lanes aligned at a fixed x -coordinate; sheets are therefore indexed by x .

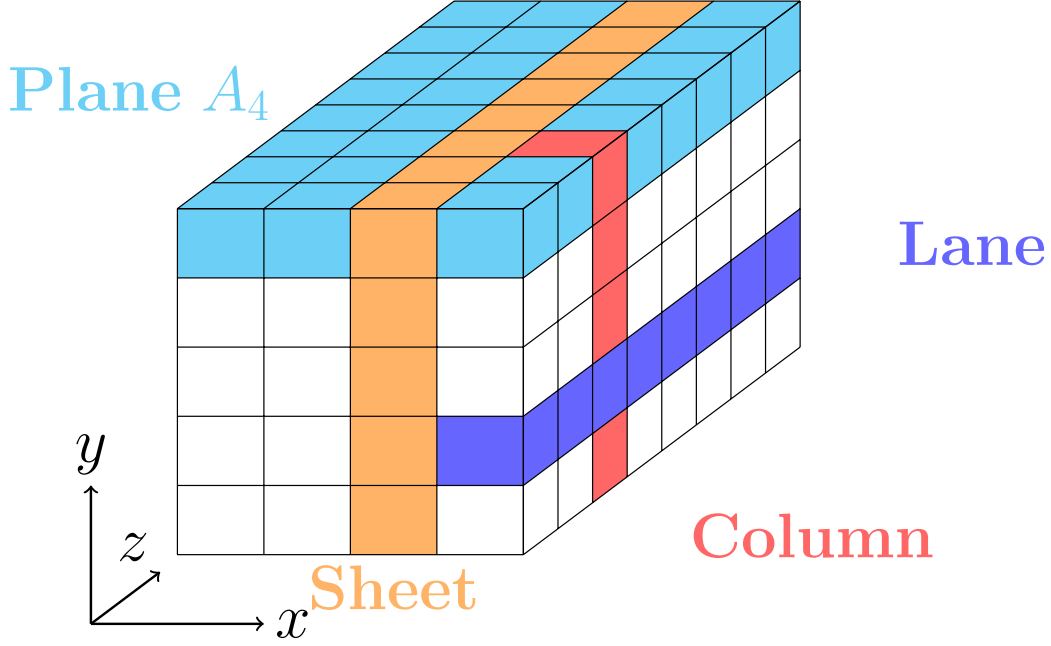


Figure 1.1: State Representation of ZC-1280

S-Box χ ZC-1280 uses a 5-bit input, 5-bit output χ function. For each bit position $i \in \{0, 1, \dots, 4\}$, the output is defined as:

$$y_i = x_i \oplus (\neg x_{i+1} \wedge x_{i+2}),$$

where all indices are taken modulo 5, or

$$S[x][y][z] \leftarrow S[x][y][z] \oplus ((\neg S[x][y+1][z]) \wedge S[x][y+2][z])$$

described in state manner.

This S-box is a 5-bit nonlinear mapping with algebraic degree 2, maximum differential probability 2^{-2} , and maximum linearity bias 2^{-2} . It is a bijection over $\mathbb{F}_2^5$, and each output bit depends on exactly 3 input bits, only two of which contribute nonlinearly. As a consequence, both the differential and linear branch numbers of χ equal 2. The bitsliced implementation requires few, well-pipelineable instructions, one AND, one NOT, and one XOR. Its quadratic algebraic structure makes it amenable to threshold implementations and masking schemes, while its compact and regular description ensures high parallelisability and efficiency in both software and hardware.

Column Parity Mixer θ The column parity mixer θ operates on the $4 \times 5 \times 64$ state of ZC-1280, organised as 5 planes, each plane contains 4 64-bit lanes. For each plane $A_y, y \in \{0, 1, 2, 3, 4\}$, the column parity is first computed as:

$$P = A_0 \oplus A_1 \oplus A_2 \oplus A_3 \oplus A_4$$

and an effect plane is derived as:

$$\begin{aligned} E[x, z] &= P \lll (1, 5) \oplus P \lll (1, 14) \\ &= P[x - 1, z - 5] \oplus P[x - 1, z - 14], \end{aligned}$$

where $\lll (a, b)$ denotes a cyclic left rotation by a bits in x-ray and b bits in z-ray, and all x indices are taken modulo 4, all z indices are taken modulo 64. The state is then updated as:

$$A_y = A_y \oplus E, \quad \text{for all } y \in \{0, 1, 2, 3, 4\}.$$

The θ mapping ensures inter-column diffusion. It can also be represented by

$$S[x][y][z] \leftarrow S[x][y][z] + \sum_{y'=0}^4 S[x-1][y'][z-5] + \sum_{y'=0}^4 S[x-1][y'][z-14].$$

It contributes to the lower bound on the weight of differential and linear trails across rounds.

Plane Shift ρ The plane shift ρ applies an intra-state rotation to introduce asymmetry between the planes before the nonlinear layer. It is defined as:

$$\begin{aligned} A_1 &\leftarrow A_1 \lll (1, 5), \\ A_2 &\leftarrow A_2 \lll (3, 24), \\ A_3 &\leftarrow A_3 \lll (0, 1), \\ A_4 &\leftarrow A_4 \lll (2, 3), \end{aligned}$$

where the first plane A_0 is left unchanged. It can also be represented by

$$S[x][y][z] = \begin{cases} S[x][y][z] & \text{if } y = 0 \\ S[x-1][y][z-5] & \text{if } y = 1 \\ S[x-3][y][z-24] & \text{if } y = 2 \\ S[x][y][z-1] & \text{if } y = 3 \\ S[x-2][y][z-3] & \text{if } y = 4 \end{cases}$$

Round Constant Addition ι The round constant addition ι injects a round-dependent constant into a single lane of the first plane to break the rotational symmetry of the other mappings. For round i , it is defined as

$$A_0[0] \leftarrow A_0[0] \oplus c_i,$$

where $c_i \in \mathbb{F}_2^{64}$ is the i -th round constant, $A_0[0]$ denotes the lane at position $x = 0$ in the bottom plane, and all other lanes of the state are left unchanged. The round index i ranges over $\{-11, \dots, 0\}$ across the $n_r = 12$ rounds of ZC-m. Without ι , the round function would commute with the lane-translation symmetry of θ , ρ , and χ , enabling trivial structural distinguishers. The round constants $\{c_i\}$ are specified in Table 1.1.

Together, ι , θ , and ρ constitute the full linear layer of ZC-1280. Their combined action ensures that any difference or linear mask introduced in one part of the state is rapidly spread across the entire state, providing the diffusion necessary to bound the weight of differential and linear trails over multiple rounds.

1.1.2 Basic Components of ZC-1536

State The state size of ZC-1536 is 1536 bits, which can be organized into 3 equally sized horizontal *planes*, each plane consists of parallel 64-bit *lanes*, as shown in Figure 1.2. Equivalently, the ZC-1536 state may be viewed as a collection of 512 *columns* of 3 bits, arranged in a 8×64 array. Planes are indexed by the coordinate y , with plane $y = 0$ at the bottom and plane $y = 2$ at the top. Within each lane, bits are indexed by z . Lanes within a plane are indexed by x , so the position of a lane in the state is uniquely identified by the coordinates (x, y) . Consequently, each bit of the state is indexed by the triplet (x, y, z) , while columns are indexed by (x, z) . We further define *Sheets* as vertical stacks of 3 lanes aligned at a fixed x -coordinate; sheets are therefore indexed by x .

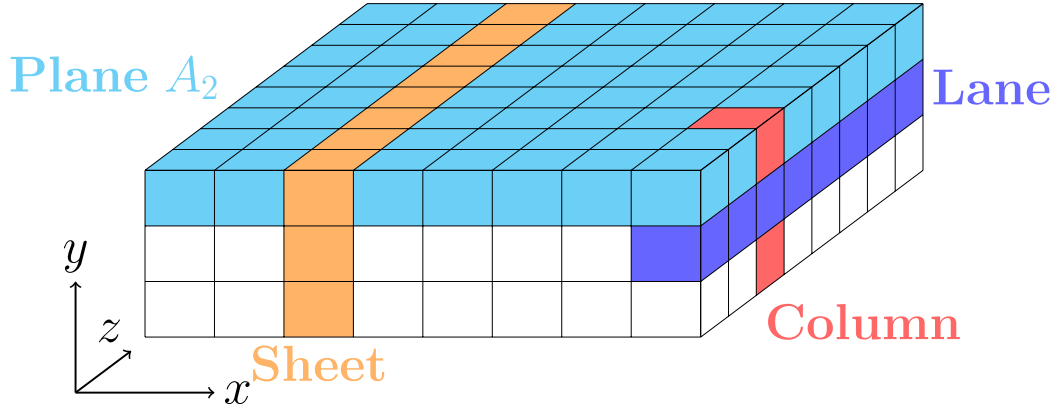


Figure 1.2: State Representation of ZC-1536

S-Box χ ZC-1536 uses a 3-bit input, 3-bit output χ function. For each bit position $i \in \{0, 1, \dots, 2\}$, the output is defined as:

$$y_i = x_i \oplus (\neg x_{i+1} \wedge x_{i+2}),$$

where all indices are taken modulo 3.

This S-box is a 3-bit nonlinear mapping with algebraic degree 2, maximum differential probability 2^{-2} , and maximum linearity bias 2^{-2} . It is a bijection over $\mathbb{F}_2^3$, and each output bit depends on all 3 input bits, only two of which contribute nonlinearly. As a consequence, both the differential and linear branch numbers of χ equal 2. The bitsliced implementation requires few, well-pipelined instructions, one AND, one NOT, and one XOR. Its quadratic algebraic structure makes it amenable to threshold implementations and masking schemes, while its compact and regular description ensures high parallelisability and efficiency in both software and hardware.

Column Parity Mixer θ The column parity mixer θ operates on the $8 \times 3 \times 64$ state of ZC-1536, organised as 3 planes, each plane contains 8 64-bit lanes. For each plane $A_y, y \in \{0, 1, 2\}$, the column parity is first computed as:

$$P = A_0 \oplus A_1 \oplus A_2$$

and an effect plane is derived as:

i	c_i	i	c_i	i	c_i
-11	0x0000000000000058	-7	0x0000000000000060	-3	0x00000000000000F0
-10	0x0000000000000038	-6	0x0000000000000120	-2	0x00000000000001A0
-9	0x000000000000003C0	-5	0x0000000000000014	-1	0x00000000000000380
-8	0x00000000000000D0	-4	0x000000000000002C	0	0x0000000000000012

Table 1.1: The round constants c_i of ZC-1280 and ZC-1536 for $-11 \leq i \leq 0$ are expressed in hexadecimal notation, with the least significant bit located at $z = 0$.

$$E[x, z] = P \lll (1, 20) \oplus P \lll (1, 56) = P[x - 1, z - 20] \oplus P[x - 1, z - 56],$$

where $\lll (a, b)$ denotes a cyclic left rotation by a bits in x-ray and b bits in z-ray, and all x indices are taken modulo 8, all z indices are taken modulo 64. The state is then updated as:

$$A_y = A_y \oplus E, \quad \text{for all } y \in \{0, 1, 2\}.$$

The θ mapping ensures inter-column diffusion. It contributes to the lower bound on the weight of differential and linear trails across rounds.

Plane Shift ρ The plane shift ρ applies an intra-state rotation to introduce asymmetry between the planes before the nonlinear layer. It is defined as:

$$\begin{aligned} A_1 &\leftarrow A_1 \lll (3, 3), \\ A_2 &\leftarrow A_2 \lll (5, 0), \end{aligned}$$

where the first plane A_0 is left unchanged.

Round Constant Addition ι The round constant addition ι injects a round-dependent constant into a single lane of the first plane to break the rotational symmetry of the other mappings. For round i , it is defined as

$$A_0[0] \leftarrow A_0[0] \oplus c_i,$$

where $c_i \in \mathbb{F}_2^{64}$ is the i -th round constant, $A_0[0]$ denotes the lane at position $x = 0$ in the bottom plane, and all other lanes of the state are left unchanged. The round index i ranges over $\{-11, \dots, 0\}$ across the $n_r = 12$ rounds of ZC-m. Without ι , the round function would commute with the lane-translation symmetry of θ , ρ , and χ , enabling trivial structural distinguishers. The round constants $\{c_i\}$ are specified in Table 1.1.

Together, ι , θ , and ρ constitute the full linear layer of ZC-1536. Their combined action ensures that any difference or linear mask introduced in one part of the state is rapidly spread across the entire state, providing the diffusion necessary to bound the weight of differential and linear trails over multiple rounds.

1.1.3 Round Function

A single round of ZC-m, illustrated for the 1280-bit and 1536-bit variants in Algorithms 1 and 2 respectively, consists of the following five steps applied in order:

1. **Round Constant Addition (ι):** A round-dependent constant c_i (listed in Table 1.1) is XORed into a single lane of the first plane A_0 . The constants break the symmetry between rounds and ensure that no round is the identity mapping.
2. **Nonlinear Layer (χ):** The χ mapping is applied columnwise to every column of the state. In ZC-1280, χ acts on 5-bit columns; in ZC-1536, on 3-bit columns. In both variants χ is the only source of nonlinearity in the round function.
3. **Plane Shift (ρ_1):** Each plane other than A_0 is rotated by a fixed offset; the first plane A_0 is left unchanged. The exact offsets depend on the variant and are given in Algorithms 1 and 2.
4. **Column Parity Mixer (θ):** The column parity is computed by XORing the lanes of each column, and an effect plane is derived by XORing two cyclically rotated copies of the parity plane. The effect plane is then added to every plane of the state, providing inter-column diffusion.
5. **Plane Shift (ρ_2):** The same plane shift as in step 3 is applied a second time, with identical rotation offsets and A_0 again left unchanged.

The full permutation ZC- m is the composition of $n_r = 12$ rounds, with round constants indexed by $i = -11, \dots, 0$.

1.2 The Construction SPONGE-DM^c

As explained in CRYPTO 2026 paper [1], we can see that the generic preimage and second preimage attacks require the evaluation of the inverses of the permutations in the absorbing phase. Therefore, we expect that replacing the permutations in the absorbing phase by random transformations may increase the security bound with respect to preimage and second preimage attacks. In fact, Foekens showed that if the permutations in the sponge are replaced by random transformations $\mathcal{F} : \mathbb{F}_2^b \rightarrow \mathbb{F}_2^b$, the security levels of the resulting construction for preimage and second preimage attacks are about n -bit and $\min\{n, c - \log_2(\alpha)\}$ -bit respectively, where n is the size of the digest, c is the capacity and $\alpha \cdot r$ is the bit length of the longest message allowed to be processed [2].

Unfortunately, whereas cryptographic permutations are in abundance, random transformations are much more scarce (and the few that exist are typically dedicated designs inspired by existing permutation-to-transformation conversions [3, 4]). Yet, we can build functions $\mathcal{F} : \mathbb{F}_2^b \rightarrow \mathbb{F}_2^b$ on top of permutations over $\mathbb{F}_2^b$ that behave sufficiently well for our purpose.

SPONGE-DM^c We introduce the function, namely SPONGE-DM^c that are entirely permutation-based and achieve improved preimage and second preimage resistance. The construction leave the permutation $f : \mathbb{F}_2^b \rightarrow \mathbb{F}_2^b$ in the squeezing phase unchanged (see Figure 1.4b), but replace it in the absorption phase by a one-way function $\mathcal{F} : \mathbb{F}_2^b \rightarrow \mathbb{F}_2^b$.

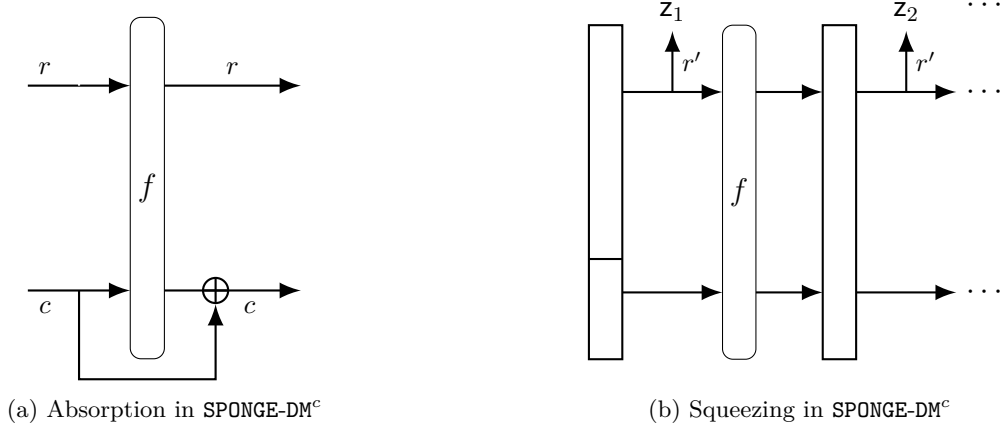
In SPONGE-DM^c, we instantiate $\mathcal{F}$ as the Davies-Meyer construction [5], or basically f with a feed-forward:

$$\mathcal{F}(x) = f(x) \oplus 0^r \parallel [x]^c. \quad (1.1)$$

The construction is depicted in Figure 1.4a.

Algorithm 1: ZuD-1280	Algorithm 2: ZuD-1536
Parameters: Number of rounds n_r 1 for Round index i from $1 - n_r$ to 0 do 2 $A = p(A)$ 3 Here p is specified by the following sequence of steps: 4 ι : 5 $A_0 \leftarrow A_0 + c_i$ 6 χ : 7 $B_0 \leftarrow \bar{A}_1 \cdot A_2$ 8 $B_1 \leftarrow \bar{A}_2 \cdot A_3$ 9 $B_2 \leftarrow \bar{A}_3 \cdot A_4$ 10 $B_3 \leftarrow \bar{A}_4 \cdot A_0$ 11 $B_4 \leftarrow \bar{A}_0 \cdot A_1$ 12 $A_y \leftarrow A_y + B_y$ for 13 $y \in \{0, 1, 2, 3, 4\}$ 14 ρ : 15 $A_1 \leftarrow A_1 \lll (1, 5)$ 16 $A_2 \leftarrow A_2 \lll (3, 24)$ 17 $A_3 \leftarrow A_3 \lll (0, 1)$ 18 $A_4 \leftarrow A_4 \lll (2, 3)$ 19 θ : 20 $P \leftarrow A_0 + A_1 + A_2 + A_3$ 21 $+ A_4$ 22 $E \leftarrow P \lll (1, 5)$ 23 $+ P \lll (1, 14)$ 24 $A_y \leftarrow A_y + E$ for 25 $y \in \{0, 1, 2, 3, 4\}$ 26 ρ : 27 $A_1 \leftarrow A_1 \lll (1, 5)$ 28 $A_2 \leftarrow A_2 \lll (3, 24)$ 29 $A_3 \leftarrow A_3 \lll (0, 1)$ 30 $A_4 \leftarrow A_4 \lll (2, 3)$	Parameters: Number of rounds n_r 1 for Round index i from $1 - n_r$ to 0 do 2 $A = p(A)$ 3 Here p is specified by the following sequence of steps: 4 ι : 5 $A_0 \leftarrow A_0 + c_i$ 6 χ : 7 $B_0 \leftarrow \bar{A}_1 \cdot A_2$ 8 $B_1 \leftarrow \bar{A}_2 \cdot A_0$ 9 $B_2 \leftarrow \bar{A}_0 \cdot A_1$ 10 $A_y \leftarrow A_y + B_y$ for 11 $y \in \{0, 1, 2\}$ 12 ρ : 13 $A_1 \leftarrow A_1 \lll (3, 3)$ 14 $A_2 \leftarrow A_2 \lll (5, 0)$ 15 θ : 16 $P \leftarrow A_0 + A_1 + A_2$ 17 $E \leftarrow P \lll (1, 20)$ 18 $+ P \lll (1, 56)$ 19 $A_y \leftarrow A_y + E$ for 20 $y \in \{0, 1, 2\}$ 21 ρ : 22 $A_1 \leftarrow A_1 \lll (3, 3)$ 23 $A_2 \leftarrow A_2 \lll (5, 0)$

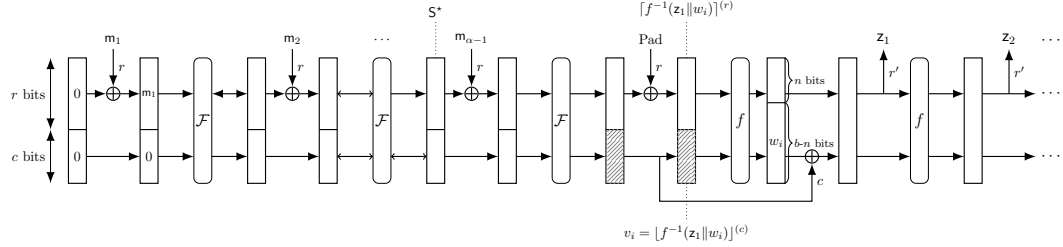
Figure 1.3: Definitions of ZC-1280 and ZC-1536 with n_r the number of rounds.

Figure 1.4: The absorption and squeezing functions of SPONGE-DM^c

The idea to place a feed-forward around the permutation is, arguably, the simplest way of adding non-invertibility to the permutation, and indeed suffices to achieve comparable indistinguishability and collision resistance as the sponge, but to obtain improved preimage and second preimage resistance.

To see this, we present below a best-known generic (second) preimage attack.

Consider the SPONGE-DM^c construction given in Figure 1.5, where $\mathcal{F}$ is instantiated with the SPONGE-DM^c compression function, while only feed-forwarding the inner part.

Figure 1.5: Generic preimage attack on the SPONGE-DM^c construction.

When $r' \geq n$, the digest consists only z_1 , where $\|z_1\|_{\mathbb{F}_2} = n$. In case $r \geq n$, we randomly select 2^t strings $w_i \in \mathbb{F}_2^{b-n}$ and concatenate each with z_1 to form a set of b -bit candidate states:

$$\mathbf{state}_i = z_1 \parallel w_i, \quad i \in \{1, 2, \dots, 2^t\}.$$

While in case $r < n$, we have to try 2^{n-r} to get one valid $\mathbf{state}_i$ satisfying the digest. For each candidate state, we compute $v_i = \lfloor f^{-1}(\mathbf{state}_i) \rfloor^{(c)}$. Next, we search for a message $\mathbf{m}$ such that

$$\lfloor \mathcal{F}(\mathbf{state}^* \oplus (\mathbf{m}_{\alpha-1} \parallel 0^c)) \rfloor^{(c)} = v_j.$$

The expected number of trials required to find such a state $\mathbf{m}$ is approximately 2^{c-t} . If $\mathbf{m}$ is found, then $\mathbf{m}_1 \parallel \mathbf{m}_2 \parallel \dots \parallel \mathbf{m}_{\alpha-1} \parallel \lfloor \mathcal{F}(\mathbf{state}^* \oplus (\mathbf{m}_{\alpha-1} \parallel 0^c)) \oplus (f^{-1}(\mathbf{state}_j)) \rfloor^{(r)}$ can produce the digest z_1 , regardless of the padding rule. The time complexity is $2^t + 2^{c-t}$ in

case $r \geq n$ and $2^{n-r+t} + 2^{c-t}$ in case $r < n$, which is minimized to

$$\begin{cases} 2^{\frac{c}{2}} & \text{if } \min\{r, r'\} \geq n, \\ 2^{\frac{n-r+c}{2}} & \text{if } r' \geq n > r. \end{cases}$$

or

$$2^{\frac{c+\max\{n-r, 0\}}{2}} \quad (1.2)$$

When $r' < n$, the process above could be repeated approximately $2^{n-r'}$ times to expect a full n -bit digest matching. Thus, the time complexity scales accordingly:

$$\begin{cases} 2^{n-r'+\frac{c}{2}} & \text{if } r \geq n > r', \\ 2^{n-r'+\frac{n-r+c}{2}} & \text{if } n > \max\{r, r'\}. \end{cases}$$

or

$$2^{\frac{c+\max\{n-r, 0\}}{2} + n - r'} \quad (1.3)$$

However, if we set the repetition in the first phrase, the complexity is $2^{t+n-r'} + 2^{c-t}$ in case $r \geq r'$ and $2^{t+n-r} + 2^{c-t}$ in case $r < r'$. Hence, the total complexity of this (second) preimage attack is given by

$$2^{\frac{n+\max\{n-2\min\{r, r'\}, c-\min\{r, r'\}\}}{2}}. \quad (1.4)$$

which is always better than 1.3, so equation 1.2 and 1.4 are the best complexity for $r' \geq n$ and $r' < n$.

Therefore, SPONGE-DM^c can be adjusted to work with a significantly reduced feed-forward size when $r > r' = c - n$ and $c > n$.

1.3 The Hash Functions

After defining the permutations ZC-m and the construction SPONGE-DM^c, we build the hash function family

$$\text{ZC-DMC-m-n} = \text{ZC-m-DM}^c[c = n + 64, n, r'],$$

where $\text{ZC-m} \in \{\text{ZC-1280}, \text{ZC-1536}\}$ and the digest length n ranges over $\{512, 768, 1024\}$. Each instance is parameterized by the state size m , the digest length n , the capacity $c = n + 64$, and the absorption rate $r' = m - c$. In addition, the squeezing rate r' is now an independent parameter, decoupled from the absorption rate r . The standard instances of the family use $r' = 64$ and are listed in Table 1.2.

For example, for 512-bit security and digest length, we have

$$\begin{aligned} \text{ZC-1280-DM}^c[576, 512, r' = 64], \\ \text{ZC-1536-DM}^c[576, 512, r' = 64]. \end{aligned}$$

For 768-bit security and digest length, we have

$$\begin{aligned} \text{ZC-1280-DM}^c[832, 768, r' = 64], \\ \text{ZC-1536-DM}^c[832, 768, r' = 64]. \end{aligned}$$

For 1024-bit security and digest length, we have

$$\begin{aligned} \text{ZC-1280-DM}^c[1088, 1024, r' = 64], \\ \text{ZC-1536-DM}^c[1088, 1024, r' = 64]. \end{aligned}$$

Table 1.2: Standard instances of the ZC-DMC-m-n family based on SPONGE-DM^c.

Instance	m	n	c	r	r'	Sec. level
ZC-1280-DM ^c [576, 512, 64]	1280	512	576	704	64	512
ZC-1536-DM ^c [576, 512, 64]	1536	512	576	960	64	512
ZC-1280-DM ^c [832, 768, 64]	1280	768	832	448	64	768
ZC-1536-DM ^c [832, 768, 64]	1536	768	832	704	64	768
ZC-1280-DM ^c [1088, 1024, 64]	1280	1024	1088	192	64	1024
ZC-1536-DM ^c [1088, 1024, 64]	1536	1024	1088	448	64	1024

Compared with the SPONGE-DM and SPONGE-EDM^c families, the SPONGE-DM^c family decouples the squeezing rate r' from the absorption rate r . The standard instances above fix $r' = 64$, which matches the security gap $c - n = 64$ between the capacity and the digest length and is the conservative choice for long messages. When the message is long, the absorption phase already requires many calls to ZC-m, so the additional squeezing-phase calls induced by a small r' contribute only a small fraction of the total cost. For applications dominated by short messages, a larger r' (paired with a smaller r , i.e. a larger c) reduces the number of squeezing calls and improves overall throughput at the same security level; such tradeoffs are discussed at the end of this section.

The absorption mapping F For an m -bit input $X = X_r \parallel X_c$, with X_r the leading r bits and X_c the trailing c bits, the absorption mapping is

$$F : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^m, \quad F(X) = \text{ZC-m}(X) \oplus (0^r \parallel X_c).$$

That is, after applying the full permutation ZC-m, the capacity portion of the input is fed forward into the corresponding bits of the output. The mapping F is a Davies-Meyer style transformation restricted to c feed-forward bits, and is one-way rather than invertible. The squeezing phase, by contrast, uses ZC-m unchanged without any feed-forward.

Padding The message `msg` is first appended with the two-bit suffix `01`, yielding $M \parallel 01$. A multi-rate padding string of the form 10^*1 is then appended so that the total length is a multiple of the rate r . Since 10^*1 has length at least 2, the number of message blocks is

$$\alpha = \left\lceil \frac{|M| + 4}{r} \right\rceil,$$

and at least two padding bits are always appended. In particular, a full extra block is appended whenever $|M| + 2 \equiv 0 \pmod{r}$ or $|M| + 2 \equiv r - 1 \pmod{r}$.

Initialization and instance separation For the standard instances in Table 1.2, the initial state depends on the digest length n . Define

$$I_{m,n} = 0^{m-3} \parallel d_n, \quad d_n = \begin{cases} 001, & n = 512, \\ 010, & n = 768, \\ 100, & n = 1024. \end{cases}$$

Before absorbing the first message block, the m -bit state is initialized to $I_{m,n}$. Thus, each initial state contains exactly one nonzero bit, and the distinguishing bits occupy the final

three positions of the serialized state. These positions belong to the capacity of every standard instance.

The instance-dependent initialization separates the standard digest-length variants sharing the same permutation. In particular, their initial states differ in capacity positions that cannot be modified by message injection. Consequently, the first permutation inputs cannot coincide across these variants, preventing the equal-first-input alignment required by the three-block cross-rate construction reported in <https://ngcc.dev/reports/hash-31.html#hash-31-1-cross-domain-distinguisher-between-zc-1536-512-and-zc-1536-768>. This modification requires no additional permutation evaluation.

Absorption Let $M_0, M_1, \dots, M_{\alpha-1}$ denote the padded message blocks, each of length r . We use S^i to denote the state input to the i -th call of F . The state evolves according to

$$\begin{aligned} S^0 &= I_{m,n} \oplus (M_0 \parallel 0^c), \\ S^i &= F(S^{i-1}) \oplus (M_i \parallel 0^c), \quad i = 1, \dots, \alpha - 1. \end{aligned}$$

After the last message block $M_{\alpha-1}$ has been processed, the absorption phase ends with the state S^α

$$S^\alpha = F(S^{\alpha-1}),$$

from which the digest is squeezed.

Squeezing. Let $\langle S \rangle_{r'}$ denote the leading r' bits of an m -bit state S . Starting from S^α , the squeezing phase extracts r' bits per iteration and applies ZC-m between iterations:

$$S^{\alpha+j} = \text{ZC-m}(S^{\alpha+j-1}), \quad j = 1, 2, \dots, t,$$

where $t = \lceil n/r' \rceil - 1$. The output stream is

$$Z = \langle S^\alpha \rangle_{r'} \parallel \langle S^{\alpha+1} \rangle_{r'} \parallel \dots \parallel \langle S^{\alpha+t} \rangle_{r'},$$

and the digest is the leading n bits of Z . When $r' \nmid n$, the final extracted block is truncated so that $|Z|$ matches n exactly; when $r' \geq n$, only the first r' bits of S^α are used and no further permutation call is made. The full procedure is summarized in Algorithm 3.

Fixed parameters for all message lengths. Each instances use the parameter settings specified in Table 1.2 for every supported message length. For a permutation width m and digest length n , the capacity, absorption rate, and squeezing rate are fixed as

$$c = n + 64, \quad r = m - n - 64, \quad r' = 64.$$

These parameters are independent of the message length, including for short and empty messages. Digest generation therefore requires $n/64 - 1$ additional permutation evaluations after absorption: 7, 11, and 15 evaluations for 512-, 768-, and 1024-bit digests, respectively. This uniform parameter policy simplifies implementation and interoperability by eliminating message-length-dependent parameter selection, while accepting the associated squeezing overhead for short messages.

Algorithm 3: $\text{ZC-DMC-m-n}[c, n, r'](M)$ based on SPONGE-DM^c

Input: message $M \in \{0, 1\}^*$; parameters c, n, r' with rate $r = m - c$

Output: n -bit digest H

```

1  /* Padding                                                    */
2   $M \leftarrow M \parallel 01$ ;
3   $j \leftarrow$  smallest  $j \geq 0$  such that  $|M| + 2 + j \equiv 0 \pmod{r}$ ;
4   $M \leftarrow M \parallel 1 \parallel 0^j \parallel 1$ ;
5  parse  $M$  as  $M_0 \parallel M_1 \parallel \dots \parallel M_{\alpha-1}$  with  $|M_i| = r$ ;
6  /* Absorption                                                    */
7   $S \leftarrow I_{m,n} \oplus (M_0 \parallel 0^c)$ ;
8  for  $i \leftarrow 1$  to  $\alpha - 1$  do
9       $S \leftarrow F(S)$ ;                                     //  $F(X) = \text{ZC-m}(X) \oplus (0^r \parallel X_c)$ 
10      $S \leftarrow S \oplus (M_i \parallel 0^c)$ ;                // absorb  $M_i$ 
11  $S \leftarrow F(S)$ ;                                       // final DM mapping
12 /* Squeezing                                                    */
13  $Z \leftarrow$  leading  $r'$  bits of  $S$ ;
14 while  $|Z| < n$  do
15      $S \leftarrow \text{ZC-m}(S)$ ;
16      $Z \leftarrow Z \parallel$  (leading  $r'$  bits of  $S$ );
17 return leading  $n$  bits of  $Z$ ;
```

Chapter 2

Design Rationale

We draw design insights from the permutations used in Keccak, Xoodoo, and Ascon. The round function should combine a compact nonlinear χ function for confusion with a strong linear layer for diffusion. For diffusion, we favor the approaches used in Keccak and Xoodoo, as they scale well to large permutation sizes such as 1536-bit and 1280-bit states. For instance, a 1536-bit state can be viewed as $1536 = 3 \times 512$. However, directly mapping this into a 2D structure and applying 512-bit rotations and XOR operations is less efficient. Instead, we adopt a 3D structure: each 512-bit segment is further divided into 8×64 -bit lanes. This enables the core linear layer to operate in a plane-wise manner. Consequently, the θ step closely resembles that of Keccak and Xoodoo, performing plane-wise XOR operations followed by lane-wise rotations, which are computationally efficient. After defining θ (linear diffusion) and χ (nonlinear confusion) as the core components, additional linear operations are required to strengthen resistance against differential and linear cryptanalysis. In a Xoodoo-like θ , where operations are naturally expressed plane-wise, a key challenge is ensuring that active columns propagate across different columns before and after the θ step. To address this, we introduce a ρ transformation applied both before and after θ . This ensures that bits originally in the same column are redistributed into different columns. A simple and efficient implementation of ρ is to permute lanes. Since lane movement is low-cost and effective, we define ρ such that the bottom plane remains unchanged, while the other planes apply distinct lane shifts. However, a search restricted to lane shifts alone yielded insufficient diffusion: differences may remain confined in a small slice for several rounds, resulting in a suboptimal lower bound on the number of active χ instances over three rounds. To avoid such confinement while keeping the implementation lightweight, we enrich ρ with lane-wise rotations. In our design, the same ρ operation is applied before and after θ , so that columns are redistributed both before the parity mixing and after it.

2.1 Design of the Construction

Besides the underlying permutations ZC-1280 and ZC-1536, the security and efficiency of the proposed hash functions are determined by the mode of operation used to turn a permutation into a hash function. Our goal is to keep the design simple and permutation-only, while achieving improved preimage and second preimage resistance compared to the classical sponge at the same state size.

The starting point is the observation that the generic attacks matching the preimage and second preimage bounds of the classical sponge make essential use of the *inverse* of the under-

lying permutation. Consequently, if we replace the absorption function by a permutation-based *one-way* transformation, then these inverse-based generic attacks no longer apply directly, and one can obtain stronger bounds without changing the squeezing phase. This is the guiding principle behind the DM^c construction of Chapter 1, following the methodology of [1].

The DM^c construction instantiates the absorption function as a Davies–Meyer transform of the permutation, $\mathcal{F}(x) = f(x) \oplus 0^r \parallel [x]^c$. This is arguably the simplest way of turning an efficiently implementable permutation into a non-invertible mapping without introducing any additional primitive.

The main design rationale is that the feed-forward destroys the direct applicability of inverse-based attacks on the sponge absorption: even if an adversary could compute f^{-1} efficiently, the relation between x and $\mathcal{F}(x)$ no longer allows the adversary to “peel off” absorption steps by iterating f^{-1} in the same way as for the classical sponge. At the same time, the squeezing phase is left unchanged and continues to benefit from the well-studied security properties of the sponge methodology (in particular for collision resistance and indifferentiability-type guarantees under standard assumptions on the permutation).

From an efficiency perspective, DM^c retains the simplicity of the sponge: the absorption and squeezing both require one permutation call per processed block, and the additional cost of the feed-forward is negligible (a single XOR on the c -bit of the state).

2.2 Design of the Permutation

This section explains the design rationale behind the permutations ZC-1280 and ZC-1536 specified in Chapter 1. The round function follows an SPN-like structure with round-constant injection, a compact non-linear layer applied column-wise, and a linear layer that provides fast diffusion across the three-dimensional state representation.

2.2.1 Choice of the Round Constants

Round constants are added in the ι step to break symmetries across rounds and to prevent structural properties that could otherwise arise from the high regularity of the round function. In particular, the remaining steps (χ , ρ , and θ) are translation-invariant with respect to XORing a constant to the state; without ι , this can facilitate slide-like distinguishers and other self-similarity phenomena in iterative constructions.

In ZC-m, the round constant c_i is XORed into the first plane A_0 (see the round function in Chapter 1). Injecting constants into a fixed, small portion of the state keeps the overhead minimal while still breaking the round symmetry globally, since the following χ and linear layer quickly spread the influence of c_i to the full state.

The concrete values of c_i for $-11 \leq i \leq 0$ are listed in Table 1.1. We intentionally choose simple, structured constants so that their origin is transparent. This avoids “hidden randomness” in the design while still providing the necessary asymmetry between rounds.

2.2.2 Choice of the Substitution Layer

The only non-linear component of the round transformation is the χ mapping. It is applied independently to each column: for ZC-1280, each column is a 5-bit vector (one bit from each of the five planes at fixed (x, z)); for ZC-1536, each column is a 3-bit vector.

We adopt the χ mapping from the Keccak family due to its compact algebraic description and its efficiency in bit-sliced implementations. In particular:

- χ is a bijection and has algebraic degree 2, which is attractive for implementing masking and related side-channel countermeasures.
- χ is highly parallelizable: all columns can be processed independently, and no lookup tables are required, which avoids cache-timing issues in software.
- The gate-level cost of a bit-sliced χ is low (essentially one AND, one NOT, and one XOR per output bit), making it suitable for both software and hardware.

We note that χ alone has limited “local” diffusion in the sense that each output bit depends only on a small number of input bits within the same column. For this reason, **ZC-m** relies on the surrounding linear layer (ρ and θ) to quickly move information across columns and planes, so that sparse differences activate many χ instances over a small number of rounds.

2.2.3 Choice of the Linear Diffusion Layer

The linear layer of **ZC-m** consists of the plane shift ρ and the column parity mixer θ . The role of θ is to provide strong inter-column diffusion through a parity-based mixing mechanism inspired by Keccak and Xoodoo: it aggregates information across planes and feeds it back to all planes with inexpensive lane-wise rotations. This construction scales well with the state size and remains efficient for both the 1280-bit and 1536-bit instances.

The role of ρ is to break the alignment of columns across steps. Since χ is applied column-wise and θ has a structured, plane-wise form, it is important that bits that start in the same column do not remain together across multiple applications of χ and θ . The pre-/post-application of ρ around θ ensures that differences and linear masks are redistributed across columns both before the parity mixing and after it.

In the design process, we observed that using only lane permutations for ρ can lead to slow diffusion and confinement effects for sparse inputs. Introducing lane-wise rotations within ρ mitigates these effects at low cost, and we select rotation offsets to provide good empirical diffusion over a small number of rounds while preserving implementation simplicity.

Chapter 3

Security Analysis

The SPONGE-DM^c construction provides the provable secure high-level architecture for permutation-based hash functions.

3.1 Security Analysis of the Construction SPONGE-DM^c

We summarize the key notation and security claims we use for the analysis of the sponge-like constructions, following [1]. Let $f : \mathbb{F}_2^b \rightarrow \mathbb{F}_2^b$ be a permutation on a b -bit state, split into an outer part of size r and an inner (capacity) part of size c with $b = r + c$. The output length is n bits, and we allow a rate r' for squeezing (hence, the digest is parsed into $\beta = \lceil n/r' \rceil$ blocks). For a message `msg`, let α denote the number of r -bit blocks after padding.

The classical sponge evaluates f during absorption and squeezing. The key observation of [1] is that generic attacks matching the sponge’s preimage and second preimage bounds make essential use of f^{-1} . Consequently, replacing the absorption primitive by a permutation-based *one-way* mapping can improve the (second) preimage bounds while preserving indistinguishability and collision resistance.

3.1.1 Dedicated attack on reduced feed-forward SPONGE-DM^c

The reduced feed-forward idea is that, since the adversary can influence the outer part via message injection, one might attempt to feed-forward only the inner part. The analysis of 1.2 shows that the variant SPONGE-DM^c could achieve a generic (second) preimage attack that is no better than brute force in relevant parameter ranges.

Concretely, the attack proceeds by guessing candidate squeezing states consistent with the digest and then matching the inner-part feed-forward constraint using an inverse-permutation step. In Equation 1.2 and Equation 1.4, and in particular obtain an overall attack complexity demonstrating that significantly reducing the feed-forward size in SPONGE-DM would not come with a substantial loss in (second) preimage resistance.

3.2 Security Evaluation of the Permutation

We have developed SAT-based automatic tools to analyze differential and linear trails of the ZC-1280 and ZC-1536 permutations. The results in below Table 3.1 indicate that the optimal 3-round differential probability is 2^{-36} , which is lower than that of the 3-round Keccak[1600] permutation (2^{-32}). Further investigations on linear trails, as well as on

differential and linear characteristics for a higher number of rounds of both ZC-1536 and ZC-1280 permutation, are currently in progress.

Permutation	Size	Type	Rounds	Prob/Cor	Reference
ZC-1280	1280	Diff	3	$\leq 2^{-35}$	Ours
		Linear	3	$\leq 2^{-16}$	Ours
ZC-1536	1536	Diff	3	$\leq 2^{-36}$	Ours
		Linear	3	$\leq 2^{-18}$	Ours
Keccak-p[1600]	1600	Diff	3	2^{-32}	[6]
		Linear	3	2^{-14}	[6]

Table 3.1: Differential probability and linear correlation comparisons for permutations ZC-1280, ZC-1536, and Keccak-p[1600].

3.3 Security Evaluation of the Hash Functions

Compared with the standard sponge construction, the feed-forward operation in SPONGE-DM^c strengthens the constrained-input constrained-output (CICO) problem. This additional constraint affects the applicability of several cryptanalytic frameworks, as discussed below for collision, preimage, and second-preimage attacks.

3.3.1 Collision Resistance

In the SPONGE-DM^c construction, the feed-forward operation strengthens the CICO problem compared with the standard sponge setting. This additional constraint affects the applicability of cryptanalytic frameworks that rely on connecting a prescribed input condition to a prescribed output or internal difference.

For standard collision resistance, however, the feed-forward operation does not directly reduce the degrees of freedom available in the input messages, and therefore does not appear to provide an additional generic security margin. Its main effect is instead on more structured collision attacks, in particular differential and internal differential collision attacks, where the feed-forward imposes additional compatibility conditions on the connector or on the internal difference.

We discuss these two types of attacks below.

Differential collision attacks. The line of differential collision attacks in [7, 8, 9, 10, 11, 12, 13] relies on the target difference algorithm (TDA) to construct a connector between the constrained input and the differential colliding trail.

For SPONGE-DM^c hash instances, however, the feed-forward introduces additional conditions on the input difference. As a result, the degrees of freedom in the input difference are consumed by these extra constraints, and the TDA can no longer generate a valid input difference by simply listing linear equations. This prevents the existing differential collision framework from being directly applied to SPONGE-DM^c.

Moreover, except for the instances targeting 512-bit preimage security, the capacity is at least 832 bits. Such a large capacity further imposes strong constraints on the connector, making it infeasible to connect the constrained input with the differential colliding trail within the existing framework.

Internal differential collision attacks. Internal differential collision attacks have been applied to SHA-3-like sponge constructions [14, 15, 16]. For SPONGE-DM^c instances, however, the feed-forward introduces additional conditions on the internal difference. These extra conditions make the cryptanalysis more constrained, and therefore more difficult than in the SHA-3 setting. Since the underlying permutation provides differential propagation strength at least comparable to that of SHA-3, we expect the resistance of SPONGE-DM^c to internal differential collision attacks to be no weaker than that of SHA-3, and potentially stronger due to the additional constraints introduced by the feed-forward.

3.3.2 Preimage Resistance

Preimage attacks on standard sponge-based hash functions, such as SHA-3 and Ascon-Hash, generally follow two research lines. The first line exploits the low-degree algebraic structure of the underlying permutation and transforms the preimage search into solving linear or low-degree equations on the input message. This line is usually referred to as the algebraic method [17] or the linear-structure method [18, 19, 20]; in the following, we refer to it as algebraic-linear preimage attacks. The second line uses meet-in-the-middle (MitM) techniques to recover preimages [21, 22, 23].

We discuss the preimage resistance of SPONGE-DM^c instances against these two types of techniques below.

Algebraic-linear preimage attacks. In this research line, the feed-forward does not directly reduce the degrees of freedom of the input messages. Moreover, since algebraic-linear attacks mainly aim to derive linear or low-degree equations on the message variables, the feed-forward at the output does not appear to significantly affect the solving efficiency of the resulting equation system. From this perspective, these preimage attacks are expected to have a difficulty comparable to those on SHA-3 and Ascon-Hash.

However, the SPONGE-DM^c instances considered here target larger security levels, namely 512/768/1024-bit security with correspondingly large capacities. These larger capacities reduce the degrees of freedom that can be used to formulate linear structures over more rounds, and therefore limit the number of rounds that can be covered by this technique.

Meet-in-the-middle preimage attacks. The feed-forward operation has a more direct impact on the applicability of MitM techniques. In MitM preimage attacks, the matching point is usually deterministic, either given by a known value or by an explicit relation. In SPONGE-DM^c , however, the feed-forward masks the actual output of the permutation. Given only the digest, the attacker cannot determine the true matching value. Consequently, the existing MitM preimage attack framework does not directly apply to SPONGE-DM^c .

3.3.3 Second-Preimage Resistance

We do not identify a dedicated second-preimage attack on the SPONGE-DM^c instances beyond the preimage attacks discussed above. Existing preimage attack frameworks would have to be adapted to the second-preimage setting, where the attacker is additionally constrained by the given target message. Therefore, the degrees of freedom available to the attacker are not larger than in the preimage setting.

For the algebraic-linear line, this additional constraint does not appear to make the construction more vulnerable than in the preimage case. For the MitM line, the feed-forward still masks the actual output of the permutation, so the deterministic matching condition

required by the existing MitM framework remains unavailable. Consequently, based on the currently known attack frameworks, we expect the second-preimage resistance of SPONGE-DM^c to be at least comparable to its preimage resistance.

3.4 Quantum Security

We next consider the security of SPONGE-DM^c in the quantum setting. The classical analysis above does not reveal any structural flaw or exploitable weak point in the bottom/inner permutation-based absorbing function. Therefore, we do not identify a meaningful attack path that would specifically exploit the internal structure of SPONGE-DM^c in quantum settings. In this situation, the natural way to assess the post-quantum security of the hash function is to model the absorbing function as an ideal function and to translate the best known classical generic attacks into their quantum generic counterparts.

Under this idealized view, we expect the quantum security of SPONGE-DM^c to follow the standard generic speedups provided by quantum search and quantum collision-finding algorithms, without any additional advantage arising from the concrete internal design.

3.4.1 Collision Resistance

In the classical ideal-function setting, the generic collision complexity is bounded by $2^{\min\{n/2, c/2\}}$. Applying the standard quantum speedup for generic collision finding yields the complexity

$$2^{\min\{n/3, c/3\}}.$$

Hence, in the absence of any structural weakness in the absorbing function, the expected quantum collision resistance of SPONGE-DM^c is

$$2^{\min\{n/3, c/3\}}.$$

3.4.2 Preimage Resistance

For preimage search, the generic classical complexity is 2^n , and Grover's algorithm reduces this to

$$2^{n/2}.$$

Since the classical analysis does not indicate any exploitable non-generic property in the absorbing function, we expect the quantum preimage resistance of SPONGE-DM^c to be

$$2^{n/2}.$$

3.4.3 Second-Preimage Resistance

Likewise, for second-preimage search, when no structural shortcut is known, the natural quantum generic attack is again Grover-type search over the message space, leading to complexity

$$2^{n/2}.$$

Therefore, under the ideal-function assumption for the absorbing function, the expected quantum second-preimage resistance of SPONGE-DM^c is

$$2^{n/2}.$$

In summary, based on the current analysis, quantum cryptanalysis of **SPONGE-DM^c** does not appear to benefit from the specific structure of the bottom/inner permutation-based absorbing function. The best known quantum attacks are therefore expected to remain generic, giving collision, preimage, and second-preimage complexities of $2^{\min\{n/3, c/3\}}$, $2^{n/2}$, and $2^{n/2}$, respectively.

3.5 Other Security Properties

3.5.1 Length Extension Attack Resistance

The sponge construction resists length extension attacks by never exposing its full internal state. Unlike Merkle-Damgård, where the digest is the full state and can be directly resumed, the sponge splits its state into a rate (output-visible) and a capacity (permanently hidden). Without the capacity bits, which are deeply entangled with the rate through a cryptographic permutation, an attacker has no valid re-entry point to extend the computation. This guarantee extends naturally to our sponge variant **SPONGE-DM^c**: as long as the rate/capacity separation is preserved, resistance to length extension is an intrinsic property of the design.

3.5.2 Zero-sum Distinguisher

Similarly to the Keccak permutation, a full-round zero-sum distinguisher can be constructed for **ZC-1280**. By fixing $1280 - 244 = 1036$ bits of the intermediate state after 5 rounds to some arbitrary value, the algebraic degree of the 5-round inverse is $3^5 = 243$, while the 7-round forward direction has degree $2^7 = 128$, together ensuring that both the input and output sets sum to zero. The resulting complexity is $O(2^{244})$ evaluations of the 5-round inverse, which normalizes to approximately $2^{244} \times 5/12 \approx 2^{242.74}$ full permutation calls, assuming that computing a round and its inverse incur equal cost. An identical argument applies to **ZC-1536**, yielding a full-round zero-sum distinguisher of the same normalized complexity $2^{242.74}$ permutation calls.

3.5.3 Statistical Randomness

We have done the statistical testing on **ZC-DMC-m-n** and the figures shows below the testing, the input is generated from 8-bytes messages and up to 10^8 bits of digest.

Table 3.2: ZC-1280 STS

C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	P-VALUE	PROPORTION	TEST
9	10	9	7	10	12	12	9	9	13	0.964255	100/100	Frequency
16	14	7	9	10	14	7	2	7	14	0.040108	100/100	BlockFrequency
13	11	7	7	12	6	10	14	7	13	0.514124	99/100	CumulativeSums
10	10	9	7	6	11	12	11	13	11	0.897763	100/100	CumulativeSums
9	5	12	8	22	7	6	14	11	6	0.004981	98/100	Runs
8	7	9	14	9	8	9	13	10	10	0.798139	99/100	LongestRun
14	8	8	11	11	9	12	10	11	6	0.851383	100/100	Rank
16	9	8	7	12	10	11	11	7	9	0.678086	96/100	FFT
14	8	8	14	11	13	12	8	5	7	0.419021	99/100	NonOverlappingTemplate
6	14	4	11	12	9	14	10	9	11	0.419021	100/100	NonOverlappingTemplate
12	13	9	10	11	7	11	15	7	5	0.494392	99/100	NonOverlappingTemplate
13	13	4	10	11	10	8	10	12	9	0.693133	99/100	NonOverlappingTemplate
10	9	10	8	7	12	11	7	12	14	0.851383	99/100	NonOverlappingTemplate
8	12	10	8	11	9	10	13	12	7	0.935716	99/100	NonOverlappingTemplate
14	9	14	8	7	10	11	11	11	5	0.595549	100/100	NonOverlappingTemplate
9	7	14	9	10	11	10	11	11	8	0.946308	99/100	NonOverlappingTemplate
13	10	10	11	11	11	12	6	6	6	0.851383	100/100	NonOverlappingTemplate
11	10	6	8	8	14	10	14	9	10	0.759756	99/100	NonOverlappingTemplate
11	7	10	14	9	3	20	5	9	12	0.014550	99/100	NonOverlappingTemplate
8	9	17	6	12	13	10	8	10	7	0.328827	100/100	NonOverlappingTemplate
11	10	12	5	8	6	12	8	13	15	0.419021	98/100	NonOverlappingTemplate
8	8	13	12	12	8	11	10	7	11	0.911413	99/100	NonOverlappingTemplate
11	13	4	6	11	5	13	12	10	15	0.181557	100/100	NonOverlappingTemplate
14	9	7	17	10	9	9	7	9	9	0.455937	100/100	NonOverlappingTemplate
11	10	7	10	8	11	6	11	12	14	0.816537	99/100	NonOverlappingTemplate
10	6	17	8	7	7	16	10	10	9	0.191687	98/100	NonOverlappingTemplate
10	11	9	12	11	8	10	14	7	9	0.911413	97/100	NonOverlappingTemplate
12	10	9	17	6	12	11	7	10	6	0.350485	99/100	NonOverlappingTemplate
4	9	8	13	6	10	13	17	13	7	0.115387	100/100	NonOverlappingTemplate
10	11	14	13	11	6	7	6	7	11	0.637119	99/100	NonOverlappingTemplate
17	10	10	10	4	15	8	8	8	8	0.249284	99/100	NonOverlappingTemplate
6	14	11	11	8	8	9	12	10	11	0.851383	99/100	NonOverlappingTemplate
8	7	8	10	8	10	11	13	11	14	0.851383	98/100	NonOverlappingTemplate
16	8	11	9	6	17	6	11	11	5	0.090936	98/100	NonOverlappingTemplate
12	9	10	8	13	7	6	8	13	14	0.616305	99/100	NonOverlappingTemplate
10	10	6	12	13	5	8	12	13	11	0.616305	100/100	NonOverlappingTemplate
6	11	14	10	9	8	11	9	10	12	0.883171	99/100	NonOverlappingTemplate
10	9	11	13	7	10	9	11	7	13	0.911413	99/100	NonOverlappingTemplate
11	9	11	11	14	6	17	7	8	8	0.249284	99/100	NonOverlappingTemplate
13	17	6	8	9	12	7	11	8	8	0.366918	98/100	NonOverlappingTemplate
4	11	7	14	19	10	9	5	14	7	0.021999	100/100	NonOverlappingTemplate
4	7	11	12	14	17	8	11	8	8	0.171867	100/100	NonOverlappingTemplate
6	8	9	11	8	14	6	15	16	7	0.171867	100/100	NonOverlappingTemplate
9	6	9	14	6	7	8	17	15	9	0.129620	97/100	NonOverlappingTemplate
5	13	11	13	9	11	8	8	14	8	0.595549	100/100	NonOverlappingTemplate
8	10	14	14	8	9	10	10	11	6	0.759756	99/100	NonOverlappingTemplate
7	11	8	8	12	10	16	6	17	0.096578	99/100	NonOverlappingTemplate	
14	10	16	10	11	6	10	8	10	5	0.366918	96/100	NonOverlappingTemplate
4	8	11	13	10	7	10	6	16	15	0.137282	100/100	NonOverlappingTemplate
10	11	16	4	13	10	7	6	11	12	0.262249	98/100	NonOverlappingTemplate
12	12	14	18	9	10	8	8	3	6	0.062821	100/100	NonOverlappingTemplate
12	14	7	13	13	6	11	8	7	9	0.554420	100/100	NonOverlappingTemplate
14	9	11	7	11	9	11	9	10	9	0.955835	97/100	NonOverlappingTemplate
11	16	7	5	11	9	14	9	12	6	0.275709	97/100	NonOverlappingTemplate
15	8	12	8	10	6	10	9	13	8	0.699313	98/100	NonOverlappingTemplate
7	5	13	8	9	11	8	7	16	16	0.145326	100/100	NonOverlappingTemplate
10	8	11	9	10	7	6	12	12	15	0.699313	98/100	NonOverlappingTemplate
12	15	16	7	5	5	6	12	7	7	0.037566	99/100	NonOverlappingTemplate
8	8	11	9	13	8	9	13	8	13	0.867692	100/100	NonOverlappingTemplate
8	6	8	15	12	11	13	7	12	8	0.534146	99/100	NonOverlappingTemplate
8	18	13	13	4	8	7	11	11	7	0.102526	100/100	NonOverlappingTemplate
9	13	8	10	15	6	10	8	15	6	0.350485	100/100	NonOverlappingTemplate
13	12	12	10	8	10	6	8	13	8	0.798139	100/100	NonOverlappingTemplate
13	8	11	8	6	11	10	7	14	12	0.699313	96/100	NonOverlappingTemplate
10	9	15	9	12	8	7	10	8	12	0.816537	98/100	NonOverlappingTemplate
6	11	14	6	8	14	12	11	8	10	0.554420	100/100	NonOverlappingTemplate
7	14	15	11	12	10	11	5	10	0.304126	100/100	NonOverlappingTemplate	
6	9	15	15	8	8	10	9	11	9	0.554420	99/100	NonOverlappingTemplate
8	20	8	16	8	11	4	9	10	6	0.016717	99/100	NonOverlappingTemplate
8	10	13	11	4	10	17	9	9	9	0.334538	100/100	NonOverlappingTemplate
7	9	5	14	11	12	4	10	15	13	0.181557	99/100	NonOverlappingTemplate
8	10	13	11	10	13	9	10	11	14	0.534146	98/100	NonOverlappingTemplate
15	14	12	12	10	8	10	4	7	8	0.334538	98/100	NonOverlappingTemplate
17	7	8	3	7	7	17	10	9	15	0.015598	98/100	NonOverlappingTemplate
4	10	15	13	9	9	9	15	7	9	0.289667	100/100	NonOverlappingTemplate
15	10	10	10	8	8	9	9	8	13	0.851383	97/100	NonOverlappingTemplate
12	9	12	10	13	8	7	11	10	10	0.935716	100/100	NonOverlappingTemplate
13	11	7	9	10	11	11	10	10	8	0.978072	99/100	NonOverlappingTemplate
9	10	13	7	9	12	8	11	11	10	0.964255	100/100	NonOverlappingTemplate
11	13	10	7	10	9	14	4	13	9	0.514124	97/100	NonOverlappingTemplate
11	12	7	9	10	9	12	16	6	8	0.574903	99/100	NonOverlappingTemplate
7	8	14	11	13	13	10	8	9	7	0.719747	98/100	NonOverlappingTemplate
14	8	8	14	11	13	12	8	5	7	0.419021	99/100	NonOverlappingTemplate
13	11	8	15	6	8	11	14	8	6	0.383827	97/100	NonOverlappingTemplate
9	16	7	7	7	11	11	7	13	12	0.455937	99/100	NonOverlappingTemplate
15	6	16	11	16	8	4	8	13	3	0.010237	99/100	NonOverlappingTemplate
12	9	9	10	12	14	8	11	6	9	0.851383	100/100	NonOverlappingTemplate
9	6	13	9	12	8	7	10	12	14	0.699313	98/100	NonOverlappingTemplate
18	12	8	13	11	10	5	8	9	6	0.171867	98/100	NonOverlappingTemplate
12	11	12	7	9	15	12	7	9	6	0.595549	99/100	NonOverlappingTemplate
9	7	8	6	16	12	8	12	11	11	0.534146	99/100	NonOverlappingTemplate
10	7	10	9	10	21	5	10	9	9	0.071177	99/100	NonOverlappingTemplate
12	17	5	8	19	10	9	7	7	6	0.019188	98/100	NonOverlappingTemplate
6	9	9	11	15	11	11	11	13	4	0.419021	99/100	NonOverlappingTemplate
(continued on right half page)												

C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	P-VALUE	PROPORTION	TEST
9	13	7	9	7	8	15	15	7	10	0.419021	100/100	NonOverlappingTemplate
13	10	9	7	6	6	11	16	11	11	0.437274	100/100	NonOverlappingTemplate
6	13	13	9	13	7	11	10	10	8	0.759756	99/100	NonOverlappingTemplate
7	12	15	10	11	11	13	8	6	7	0.554420	100/100	NonOverlappingTemplate
15	8	7	8	10	13	6	10	11	12	0.616305	98/100	NonOverlappingTemplate
11	9	8	13	10	11	9	13	5	11	0.816537	99/100	NonOverlappingTemplate
11	12	6	15	10	10	15	6	5	10	0.262249	100/100	NonOverlappingTemplate
7	8	7	8	10	13	11	9	12	15	0.678086	100/100	NonOverlappingTemplate
14	15	6	10	11	6	7	12	7	12	0.350485	100/100	NonOverlappingTemplate
12	9	4	10	11	17	10	12	8	7	0.289667	99/100	NonOverlappingTemplate
8	6	10	10	9	11	11	7	18	10	0.383827	100/100	NonOverlappingTemplate
10	2	5	7	7	10	20	10	14	15	0.003201	98/100	NonOverlappingTemplate
11	9	10	13	14	11	5	12	7	8	0.637119	99/100	NonOverlappingTemplate
10	7	8	15	13	7	13	10	7	10	0.595549	100/100	NonOverlappingTemplate
7	13	8	9	7	7	10	14	11	14	0.595549	99/100	NonOverlappingTemplate
10	10	12	10	10	11	8	8	11	10	0.967823	100/100	NonOverlappingTemplate
9	10	11	13	7	8	8	11	11	12	0.946308	99/100	NonOverlappingTemplate
9	14	14	7	9	6	14	11	12	4	0.296810	100/100	NonOverlappingTemplate
12	7	7	10	12	9	14	12	8	9	0.816537	98/100	NonOverlappingTemplate
13	13	10	10	9	7	9	14	7	8	0.759756	99/100	NonOverlappingTemplate
9												

Table 3.3: ZC-1536 STS

C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	P-VALUE	PROPORTION	TEST	(continued from left half page)												
6	11	12	8	9	6	9	9	10	20	0.558721	99/100	Frequency	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	P-VALUE	PROPORTION	TEST
11	7	10	14	15	10	6	8	12	7	0.494392	98/100	BlockFrequency	9	13	11	8	11	13	10	7	13	5	0.657033	100/100	NonOverlappingTemplate
6	9	7	8	17	9	10	10	15	9	0.304126	99/100	CumulativeSums	7	10	9	10	8	17	6	8	14	11	0.350485	97/100	NonOverlappingTemplate
6	9	9	14	8	11	12	10	7	14	0.657933	98/100	CumulativeSums	12	10	8	7	6	17	11	11	11	7	0.401199	99/100	NonOverlappingTemplate
11	13	10	14	8	8	10	7	12	7	0.779188	99/100	Runs	8	13	14	6	12	9	16	7	8	7	0.289667	100/100	NonOverlappingTemplate
7	5	12	10	11	16	7	8	13	11	0.366918	98/100	LongestRun	16	11	9	11	12	9	7	8	7	10	0.678686	98/100	NonOverlappingTemplate
7	8	6	11	9	8	12	17	11	9	0.514124	99/100	Rank	11	7	8	7	15	15	13	7	7	10	0.350485	98/100	NonOverlappingTemplate
7	8	12	10	11	9	14	10	10	9	0.935716	100/100	FFT	11	10	7	10	9	9	13	8	13	10	0.946308	100/100	NonOverlappingTemplate
10	13	14	12	10	6	8	9	6	12	0.637119	100/100	NonOverlappingTemplate	11	16	11	10	7	12	11	7	8	7	0.595549	98/100	NonOverlappingTemplate
9	8	7	9	10	18	12	8	9	10	0.455937	99/100	NonOverlappingTemplate	13	9	11	11	9	8	2	12	15	10	0.275709	99/100	NonOverlappingTemplate
11	11	10	6	8	11	8	12	12	11	0.935716	99/100	NonOverlappingTemplate	14	6	12	6	14	9	9	10	8	12	0.554420	98/100	NonOverlappingTemplate
11	7	12	10	13	11	7	8	9	12	0.897763	98/100	NonOverlappingTemplate	11	10	10	13	8	10	13	12	6	7	0.816537	96/100	NonOverlappingTemplate
15	8	10	8	10	15	9	11	8	8	0.534146	99/100	NonOverlappingTemplate	8	13	5	8	13	11	13	10	12	7	0.595549	100/100	NonOverlappingTemplate
8	6	13	12	14	10	18	7	4	8	0.062821	100/100	NonOverlappingTemplate	15	8	10	5	11	8	9	14	8	12	0.494392	99/100	NonOverlappingTemplate
6	9	12	8	10	14	8	10	12	11	0.834308	100/100	NonOverlappingTemplate	7	10	5	9	15	8	10	16	16	4	0.045675	100/100	NonOverlappingTemplate
6	17	9	4	18	11	7	8	14	6	0.011791	100/100	NonOverlappingTemplate	13	9	10	14	11	8	8	11	6	10	0.816537	99/100	NonOverlappingTemplate
16	13	7	9	5	9	12	9	7	13	0.319884	97/100	NonOverlappingTemplate	8	9	8	12	9	13	12	7	12	0.911413	100/100	NonOverlappingTemplate	
10	15	10	8	9	8	12	11	11	6	0.779188	99/100	NonOverlappingTemplate	12	6	6	16	17	15	6	8	5	9	0.023545	98/100	NonOverlappingTemplate
16	13	7	9	7	14	10	5	10	9	0.304126	100/100	NonOverlappingTemplate	10	10	11	14	8	12	8	8	8	11	0.924076	97/100	NonOverlappingTemplate
8	10	7	8	10	12	15	14	8	9	0.637119	100/100	NonOverlappingTemplate	5	9	10	6	8	16	15	12	5	14	0.085387	100/100	NonOverlappingTemplate
10	9	8	8	8	14	13	16	4	10	0.275709	100/100	NonOverlappingTemplate	11	13	8	12	6	9	8	6	16	11	0.419021	97/100	NonOverlappingTemplate
1	10	9	7	9	10	16	16	7	15	0.019188	100/100	NonOverlappingTemplate	5	4	5	12	10	12	16	15	13	8	0.051942	100/100	NonOverlappingTemplate
13	10	13	18	12	11	6	7	3	7	0.048716	99/100	NonOverlappingTemplate	10	13	8	6	9	14	9	10	12	9	0.816537	100/100	NonOverlappingTemplate
8	11	13	13	11	4	9	10	10	11	0.719747	99/100	NonOverlappingTemplate	13	6	8	8	10	18	6	9	8	14	0.143236	98/100	NonOverlappingTemplate
8	7	9	16	12	11	7	15	7	8	0.334538	100/100	NonOverlappingTemplate	9	8	10	19	12	10	8	7	9	8	0.289667	98/100	NonOverlappingTemplate
11	11	12	7	7	14	10	12	8	8	0.816537	98/100	NonOverlappingTemplate	2	17	14	11	3	10	13	11	5	14	0.006196	100/100	NonOverlappingTemplate
3	19	12	10	8	8	6	12	11	11	0.058984	100/100	NonOverlappingTemplate	10	10	10	4	11	10	15	7	13	10	0.534146	98/100	NonOverlappingTemplate
8	10	11	16	11	6	9	9	7	13	0.554420	100/100	NonOverlappingTemplate	11	19	7	4	14	10	7	9	9	10	0.080519	98/100	NonOverlappingTemplate
6	10	8	10	13	11	11	10	10	11	0.955835	100/100	NonOverlappingTemplate	14	9	12	7	11	6	11	10	9	11	0.834308	98/100	NonOverlappingTemplate
10	11	9	5	6	13	8	17	9	12	0.275709	100/100	NonOverlappingTemplate	12	11	7	12	10	12	9	9	10	8	0.971699	100/100	NonOverlappingTemplate
8	10	8	15	4	8	8	14	12	13	0.304126	99/100	NonOverlappingTemplate	7	13	10	15	10	8	10	9	8	10	0.816537	98/100	NonOverlappingTemplate
7	11	13	6	8	14	12	8	12	9	0.657933	100/100	NonOverlappingTemplate	15	5	13	7	11	8	11	12	6	12	0.366918	97/100	NonOverlappingTemplate
7	7	17	9	9	10	10	10	12	9	0.595549	99/100	NonOverlappingTemplate	9	11	9	9	12	9	10	12	10	9	0.997823	99/100	NonOverlappingTemplate
14	7	7	12	7	9	9	9	15	11	0.574903	100/100	NonOverlappingTemplate	20	6	8	14	7	12	6	10	7	10	0.042808	99/100	NonOverlappingTemplate
12	9	12	12	13	9	5	6	10	12	0.657933	98/100	NonOverlappingTemplate	11	7	9	11	9	8	12	10	11	12	0.978072	100/100	NonOverlappingTemplate
10	11	5	9	8	11	11	14	9	12	0.798139	97/100	NonOverlappingTemplate	6	17	10	10	12	15	9	12	6	3	0.058984	99/100	NonOverlappingTemplate
12	10	8	9	8	12	13	9	9	10	0.971699	98/100	NonOverlappingTemplate	13	8	11	7	10	7	11	11	9	13	0.883171	99/100	NonOverlappingTemplate
12	11	7	6	4	14	14	12	14	6	0.145326	98/100	NonOverlappingTemplate	13	8	8	11	5	13	9	10	13	10	0.719747	98/100	NonOverlappingTemplate
8	13	10	15	17	7	7	7	6	10	0.162006	97/100	NonOverlappingTemplate	6	13	13	10	10	9	16	6	14	3	0.085387	98/100	NonOverlappingTemplate
9	15	9	7	10	9	12	9	10	10	0.897763	98/100	NonOverlappingTemplate	10	14	10	6	6	11	13	9	11	10	0.739918	98/100	NonOverlappingTemplate
11	12	8	10	12	10	6	11	9	11	0.955835	100/100	NonOverlappingTemplate	12	9	11	7	11	14	8	13	10	5	0.637119	100/100	NonOverlappingTemplate
8	10	9	7	17	13	10	10	9	7	0.514124	99/100	NonOverlappingTemplate	10	8	7	17	16	4	9	10	10	9	0.137282	99/100	NonOverlappingTemplate
14	8	16	6	5	7	12	11	15	6	0.085387	97/100	NonOverlappingTemplate	9	7	8	10	11	20	7	10	9	10	0.181557	97/100	NonOverlappingTemplate
10	12	11	6	11	7	8	7	13	15	0.554420	100/100	NonOverlappingTemplate	5	9	9	13	9	15	15	12	6	7	0.236810	100/100	NonOverlappingTemplate
10	7	8	13	9	11	8	14	9	11	0.867692	99/100	NonOverlappingTemplate	6	15	8	8	9	14	12	10	6	12	0.437274	100/100	NonOverlappingTemplate
8	6	12	11	8	16	11	10	10	8	0.637119	99/100	NonOverlappingTemplate	10	7	15	9	12	8	7	11	15	6	0.401199	100/100	NonOverlappingTemplate
13	6	14	6	11	5	11	9	15	15	0.275709	100/100	NonOverlappingTemplate	9	7	8	10	11	12	11	8	9	9	0.236810	100/100	NonOverlappingTemplate
4	7	9	12	12	12	14	13	6	9	0.350485	100/100	NonOverlappingTemplate	11	12	12	11	10	8	10	13	6	7	0.851383	99/100	NonOverlappingTemplate
13	12	11	5	4	19	9	8	9	10	0.062821	100/100	NonOverlappingTemplate	12	11	11	10	10	11	7	13	5	0.834308	99/100	NonOverlappingTemplate	
14	8	5	12	9	11	9	10	14	8	0.616305	98/100	NonOverlappingTemplate	8	11	15	6	7	8	12	8	10	15	0.419021	100/100	NonOverlappingTemplate
10	13	10	12	14	5	8	10	8	10	0.719747	99/100	NonOverlappingTemplate	9	7	16	8	11	12	14	9	12	10	0.236810	100/100	NonOverlappingTemplate
8	9	12	8	12	11	10	8	14	8	0.897763	100/100	NonOverlappingTemplate	6	10	7	7	7	8	22	11	13	9	0.016717	100/100	NonOverlappingTemplate
8	6	14	7	10	16	8	9	7	15	0.213309	100/100	NonOverlappingTemplate	3	11	14	11	8	10	11	11	12	9	0.554420	99/100	NonOverlappingTemplate
6	11	10	10	12	8	6	6	17	14	0.202268	99/100	NonOverlappingTemplate	12	8	12	10	11	12	11	12	7	5	0.779188	100/100	NonOverlappingTemplate
8	4	11	15	10	14	5	11	11	11	0.275709	100/100	NonOverlappingTemplate	13	6	5	11	15	12	14	10	10	4	0.153763	100/100	NonOverlappingTemplate
8	11	11	12	6	7	8	13	16	8	0.455937	97/100	NonOverlappingTemplate	8	13	8	9	5	9	13	11	12	12	0.719747	99/100	NonOverlappingTemplate
11	9	9	12	9	10	9	11	11	9	0.998821	99/100	NonOverlappingTemplate	11	8	8	3	13	11	10	13	12	11	0.514124	98/100	NonOverlappingTemplate
5	9	17	13	14	10	7	6	6	9	0.071177	99/100	NonOverlappingTemplate	9	12	8	7	10	6	7	12	13	16	0.419021	97/100	NonOverlappingTemplate
9	13	7	16	10	11	11	6	10	7	0.514124	99/100	NonOverlappingTemplate	10	11	11	10	11	11	16	4	7	9	0.474986	100/100	NonOverlappingTemplate
8	4	12	14	15	12	13	6	9	7	0.191															

Chapter 4

Performance Evaluation

This chapter reports performance evaluation results following the submission requirements. It includes **(1) performance analysis** (explaining performance drivers from the design principles) and **(2) performance testing** (platform configuration and benchmark results on mainstream software platforms). Concrete cycle counts and throughput numbers are left as placeholders to be filled in by the implementer.

4.1 Performance analysis (design principles)

This section analyzes the main performance cost drivers from the algorithmic structure and elementary operations, and explains why high throughput can be achieved in software.

4.1.1 Elementary operations and round-function cost

Dominant cost factors of the permutation. The dominant computation of our hash instances is the **ZC-m** permutation. Each round consists primarily of: (i) constant injection (ι , a few XORs), (ii) a column-wise non-linear layer (χ , implemented in a bit-sliced manner, mainly AND/NOT/XOR), (iii) a linear diffusion layer (ρ, θ , mainly shifts/rotations and XOR). Overall, **XOR and word rotations** are the main linear operations and **AND** is the main non-linear operation, which makes the design efficient on commodity processors with native word-wise bit operations.

Parallelism and vectorization. The state is organized in multiple 64-bit lanes; χ operates independently per column, and ρ and θ are composed of lane-wise shifts/rotations and XORs. Hence, the permutation naturally supports:

- parallel bit operations on 64-bit general-purpose registers;
- SIMD parallel processing across lanes (e.g., AVX2);
- instruction-level parallelism (ILP) in software implementations.

Therefore, the software performance of **ZC-m** is mainly determined by the number of ANDs per round, the implementation of rotations/shifts, and the quality of register allocation and instruction scheduling.

4.1.2 Mode-level overhead

SPONGE-DM. All implementations evaluated in this chapter instantiate the hash function using the SPONGE-DM construction. In absorption, the Davies–Meyer feed-forward adds one XOR on the b -bit state per permutation call, which is negligible compared to the permutation itself. Thus, performance is dominated by the number of permutation calls (one per absorbed block, plus fixed overhead for initialization/finalization depending on the exact instance) and by constant factors in the permutation implementation.

4.2 Performance testing (software implementations)

This section reports benchmark results of software implementations. The primary metric is the efficiency of generating hash outputs (reported as cycles/byte and throughput in MB/s), together with a detailed description of the implementation platform and toolchain.

4.2.1 Implementation variants and platform configuration

Implementation variants. We provide three software implementation variants for both the ZC-1280-based (1280-bit state) and ZC-1536-based (1536-bit state) SPONGE-DM hash instances:

- **Generic:** portable scalar implementation.
- **AVX2-optimized:** implementation optimized for AVX2-enabled x86_64 processors.
- **Embedded 32-bit:** a 32-bit-oriented implementation intended for embedded-class CPUs.

Toolchain

- Language: C (ISO C99)
- Compiler and version: GCC 12.2.0 (Debian 12.2.0-14+deb12u1, invoked via `cc`)
- Compilation flags: `-O2 -std=c99`, `-O2 -std=c99 -mavx2` (for AVX2), `-O2 -std=c99 -Wall -Wextra` (for embedded 32-bit)

Software platform.

- OS: Debian GNU/Linux 12 (bookworm).
- CPU model: 13th Gen Intel(R) Core(TM) i9-13900
- CPU frequency (nominal/fixed): 2.0 GHz (nominal, dynamic frequency scaling enabled)
- Memory: 64 GB DDR4-3200
- ISA extensions used: AVX2 for optimized implementation
- Timing method: `rdtsc`
- Pinning / frequency control: no

4.2.2 Benchmark methodology

Our software implementations and benchmarks are driven by the organizer-provided programming interface (API) package `API_CryptHash.zip`. Concretely, we compile and link our permutation/hash code against the official API, and execute the corresponding unified batch command, which runs the mandated conformance tests and prints a standardized completion banner.

The official test harness exercises hash generation for the following message lengths (all lengths are expressed in bits):

- all messages lengths from 0 up to 2^{12} bits (inclusive), producing the corresponding digest for each length;
- 3 messages of length 2^{23} bits;
- 3 messages of length 2^{33} bits;
- a cyclic test for messages of length 2^{13} bits, producing the corresponding digest under the prescribed looping procedure.

When the batch completes successfully, the console output ends with a progress indicator and a success line of the form:

```
[#####] [100%] SUCCESS!
Files have been saved in the 'output' folder within the working directory.
Total duration: <runtime>
```

The `output/` directory then contains the corresponding hash values (and any auxiliary logs required by the harness).

4.2.3 Benchmark results

Hash performance. We report software performance of SPONGE-DM-based hashing instantiated with ZC-1280 (1280-bit) and ZC-1536 (1536-bit). Fill in seconds for each implementation variant with digest length to be 512 bits.

Instance	Variant	runtime
ZC-1280-DM ^c	Generic	33 s
ZC-1280-DM ^c	AVX2	22 s
ZC-1280-DM ^c	32-bit	44 s
ZC-1536-DM ^c	Generic	36 s
ZC-1536-DM ^c	AVX2	32 s
ZC-1536-DM ^c	32-bit	37 s

Table 4.1: Software performance of SPONGE-DM^c-based hashings with digest to be 512 bits.

Chapter 5

Feature Overview

5.1 Novelty and Comparison to Keccak-p and SHA-3

This section summarizes the main novelty of our design and compares it to the Keccak-p permutation and the SHA-3 sponge family from the perspectives of specification, security, and performance.

5.1.1 Specification and Design Choices

Large-state, 3D permutation family. We introduce a family of large permutations ZC-1280 (1280-bit) and ZC-1536 (1536-bit) that generalize Xoodoo-like design principles to a three-dimensional state representation. The round function follows a compact SPN-style structure with a column-wise χ mapping for non-linearity and a lightweight linear layer built from ρ and θ .

Simplicity and implementability. Like Keccak-p and related bit-sliced permutations, the round function is expressed using word-wise Boolean operations and rotations/shifts, avoiding lookup tables. This yields a concise specification and supports efficient software implementations in both portable scalar code and SIMD-optimized code (e.g., AVX2).

5.1.2 Security Highlights

Automated analysis of differential and linear trails. We evaluated the resistance of ZC-1280 and ZC-1536 against differential and linear cryptanalysis using automated tools, including SAT-based search. Within the explored search space and constraints, the best three-round trails found for ZC-m indicate stronger resistance than the corresponding three-round trails reported for Keccak-p under the same model.

For reference, three-round Keccak-p admits differential trails of probability 2^{-32} with 16 active χ instances, and linear trails with correlation about 2^{-14} with 13 active χ instances. In our SAT-based searches for three-round ZC-1280/ZC-1536:

- we did not find any differential trail with at most 16 active χ instances; and
- we did not find any linear trail with correlation at least 2^{-15} .

These results should be interpreted as empirical evidence (not proofs), but they support the claim that the interaction between the 3D state organization and the chosen linear layer effectively suppresses sparse, low-weight trails over few rounds.

Permutation-level security margin and comparison to SHA-3. SHA-3 derives its security from a well-studied sponge construction instantiated with `Keccak-p`. Our design keeps the same philosophy of relying on a single, well-specified permutation, but targets larger states (1280/1536) and a 3D structure to better exploit word-wise and SIMD-friendly operations while strengthening diffusion across planes and columns.

Mode-level security beyond the classical sponge. In addition to the permutation, we employ sponge-like hashing modes that improve preimage and second preimage security compared to the classical sponge at the same capacity. The underlying intuition is that generic sponge attacks matching the (second) preimage bounds rely crucially on inverse access to the permutation, and can be obstructed by using non-invertible absorption mappings.

5.1.3 Efficiency–Security Trade-off via DM^c

We provide the permutation-only absorption mapping, representing a different point in the efficiency–security design space:

- DM^c : a reduced feed-forward variant that can be more efficient in constrained settings, at the cost of degraded (second) preimage resistance under dedicated generic attacks;

Compared to the normal sponge (as used in SHA-3), these modes provide additional flexibility: for a fixed permutation size, one can obtain stronger (second) preimage guarantees, or alternatively achieve comparable guarantees with a smaller state (and thus potentially better performance).

Bibliography

- [1] Siwei Sun, Shun Li, Zhiyu Zhang, Charlotte Lefevre, Bart Mennink, Zhen Qin, and Dengguo Feng. Permutation-based hashing with stronger (second) preimage resistance - application to hash-based signature schemes. *IACR Cryptol. ePrint Arch.*, 2025:963, 2025.
- [2] Robin Foekens. *Security of the Sponge Construction with a Random Transformation*. PhD thesis, Bachelor’s Thesis, 2023.
- [3] Subhadeep Banik, Takanori Isobe, Fukang Liu, Kazuhiko Minematsu, and Kosei Sakamoto. Orthros: A low-latency prf. *IACR Transactions on Symmetric Cryptology*, pages 37–77, 2021.
- [4] Ravi Anand, Subhadeep Banik, Andrea Caforio, Tatsuya Ishikawa, Takanori Isobe, Fukang Liu, Kazuhiko Minematsu, Mostafizar Rahman, and Kosei Sakamoto. Gleeok: A family of low-latency prfs and its applications to authenticated encryption. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(2):545–587, 2024.
- [5] Stephen M Matyas. Generating strong one-way functions with cryptographic algorithm. *IBM Technical Disclosure Bulletin*, 27:5658–5659, 1985.
- [6] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Keccaktools software. <http://keccak.noekeon.org/>, 2014. Accessed: 2014.
- [7] Itai Dinur, Orr Dunkelman, and Adi Shamir. New attacks on keccak-224 and keccak-256. In Anne Canteaut, editor, *Fast Software Encryption - 19th International Workshop, FSE 2012, Washington, DC, USA, March 19-21, 2012. Revised Selected Papers*, volume 7549 of *Lecture Notes in Computer Science*, pages 442–461. Springer, 2012.
- [8] Itai Dinur, Orr Dunkelman, and Adi Shamir. Improved practical attacks on round-reduced keccak. *J. Cryptol.*, 27(2):183–209, 2014.
- [9] Kexin Qiao, Ling Song, Meicheng Liu, and Jian Guo. New collision attacks on round-reduced keccak. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology - EUROCRYPT 2017 - 36th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Paris, France, April 30 - May 4, 2017, Proceedings, Part III*, volume 10212 of *Lecture Notes in Computer Science*, pages 216–243, 2017.
- [10] Ling Song, Guohong Liao, and Jian Guo. Non-full sbox linearization: Applications to collision attacks on round-reduced keccak. In Jonathan Katz and Hovav Shacham,

- editors, *Advances in Cryptology - CRYPTO 2017 - 37th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 20-24, 2017, Proceedings, Part II*, volume 10402 of *Lecture Notes in Computer Science*, pages 428–451. Springer, 2017.
- [11] Jian Guo, Guohong Liao, Guozhen Liu, Meicheng Liu, Kexin Qiao, and Ling Song. Practical collision attacks against round-reduced SHA-3. *J. Cryptol.*, 33(1):228–270, 2020.
 - [12] Jian Guo, Guozhen Liu, Ling Song, and Yi Tu. Exploring SAT for cryptanalysis: (quantum) collision attacks against 6-round SHA-3. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part III*, volume 13793 of *Lecture Notes in Computer Science*, pages 645–674. Springer, 2022.
 - [13] Senyang Huang, Orna Agmon Ben-Yehuda, Orr Dunkelman, and Alexander Maximov. Finding collisions against 4-round SHA-3-384 in practical time. *IACR Trans. Symmetric Cryptol.*, 2022(3):239–270, 2022.
 - [14] Itai Dinur, Orr Dunkelman, and Adi Shamir. Collision attacks on up to 5 rounds of sha-3 using generalized internal differentials. In *International Workshop on Fast Software Encryption*, pages 219–240. Springer, 2013.
 - [15] Zhongyi Zhang, Chengan Hou, and Meicheng Liu. Collision attacks on round-reduced sha-3 using conditional internal differentials. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 220–251. Springer, 2023.
 - [16] Zhongyi Zhang, Chengan Hou, and Meicheng Liu. Probabilistic linearization: internal differential collisions in up to 6 rounds of sha-3. In *Annual International Cryptology Conference*, pages 241–272. Springer, 2024.
 - [17] Huina Li, Le He, Shiyao Chen, Jian Guo, and Weidong Qiu. Automatic preimage attack framework on ascon using a linearize-and-guess approach. *IACR Transactions on Symmetric Cryptology*, 2023(3):74–100, 2023.
 - [18] Jian Guo, Meicheng Liu, and Ling Song. Linear structures: Applications to cryptanalysis of round-reduced keccak. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 249–274. Springer, 2016.
 - [19] Ting Li and Yao Sun. Preimage attacks on round-reduced keccak-224/256 via an allocating approach. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 556–584. Springer, 2019.
 - [20] Xiaoen Lin, Zhengrong Lu, Yantian Shen, Chongxu Ren, Hongbo Yu, Enming Dong, and Wenhao Wu. Practical preimage attack on 5-round keccak [r= 640, c= 160]. *IACR Transactions on Symmetric Cryptology*, 2026(1):189–211, 2026.
 - [21] Lingyue Qin, Jialiang Hua, Xiaoyang Dong, Hailun Yan, and Xiaoyun Wang. Meet-in-the-middle preimage attacks on sponge-based hashing. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 158–188. Springer, 2023.

- [22] Xiaoyang Dong, Boxin Zhao, Lingyue Qin, Qingliang Hou, Shun Zhang, and Xiaoyun Wang. Generic mitm attack frameworks on sponge constructions. In *Annual International Cryptology Conference*, pages 3–37. Springer, 2024.
- [23] Lingyue Qin, Boxin Zhao, Qingliang Hou, Keting Jia, Gaoli Wang, and Xiaoyang Dong. Linear cancellations in the mitm attacks on sponge functions. *IACR Transactions on Symmetric Cryptology*, 2025(3):633–692, 2025.